TP1 : Chiffrement Affine et Algorithme d'Euclide Étendu

BAHRI ALAA

23 février 2026

1 Classe `BigIntMath`

Cette classe contient des fonctions mathématiques pour travailler avec de grands entiers et la théorie des nombres.

```
class BigIntMath:
```

Explication : Déclare la classe `BigIntMath`. Les méthodes seront statiques et peuvent être appelées sans créer une instance.

1.1 Méthode `extendedGCD`

```
@staticmethod
def extendedGCD(a, b):
    r, rr = a, b
    u, v = 1, 0
    uu, vv = 0, 1
    while rr != 0:
        q = r // rr
        r, rr = rr, r - q * rr
        u, uu = uu, u - q * uu
        v, vv = vv, v - q * vv
    return r, u, v
```

Explications ligne par ligne :

- `@staticmethod` : la méthode n'a pas besoin d'une instance.
- `r, rr = a, b` : initialise les valeurs de travail.
- `u, v = 1,0 ; uu, vv = 0,1` : initialise les coefficients de Bézout.
- `while rr != 0` : continue tant que le reste n'est pas nul.
- `q = r // rr` : calcule le quotient entier.
- Mise à jour des restes et coefficients selon l'algorithme d'Euclide étendu.
- `return r, u, v` : retourne le pgcd et les coefficients de Bézout.

1.2 Méthode `modInverse`

```
@staticmethod
def modInverse(a, n):
    d, u, v = BigIntMath.extendedGCD(a, n)
    if d != 1:
        return None
    return u % n
```

Explications :
— Appelle `extendedGCD` pour obtenir le pgcd et les coefficients.
— Si `d != 1`, a n'a pas d'inverse modulo n, retourne `None`.
— Sinon, retourne $u \mod n$, l'inverse modulaire.

2 Classe `AffineCipher`

2.1 Initialisation

```
def __init__(self, a, b, n=26):
    self.a = a
    self.b = b
    self.n = n
    self.a_inv = BigIntMath.modInverse(a, n)
```

Explications :
— `a` et `b` : clés du chiffrement affine.
— `n` : taille de l'alphabet (26 lettres).
— `self.a_inv` : inverse modulaire de `a`, utilisé pour déchiffrer.

2.2 Méthode `encrypt`

```
def encrypt(self, message):
    res = ""
    for char in message.upper():
        if 'A' <= char <= 'Z':
            m = ord(char) - ord('A')
            c = (self.a * m + self.b) % self.n
            res += chr(c + ord('A'))
        else:
            res += char
    return res
```

Explications :
— Initialise la chaîne vide `res`.
— Transforme le message en majuscules.
— Convertit chaque lettre en indice 0-25.
— Applique $C = (a \cdot M + b) \mod n$.
— Convertit l'indice en caractère et l'ajoute à `res`.
— Les caractères non alphabétiques sont conservés.

2.3 Méthode `decrypt`

```
def decrypt(self, crypte):
    res = ""
    for char in crypte.upper():
        if 'A' <= char <= 'Z':
            c = ord(char) - ord('A')
            m = (self.a_inv * (c - self.b)) % self.n
            res += chr(m + ord('A'))
        else:
```

```
            res += char
    return res
```

Explications :

— Initialise la chaîne vide `res`.
— Parcourt chaque caractère en majuscules.
— Convertit chaque lettre en indice 0-25.
— Applique $M = a^{-1} \cdot (C - b) \mod n$.
— Convertit l'indice en caractère et ajoute à `res`.
— Les caractères non alphabétiques restent inchangés.

3 Partie principale et méthode d'exécution

```
if __name__ == "__main__":
    print("␣test␣euclide␣")
    val_a = int(input("entrez␣la␣valeur␣de␣a␣:␣"))
    val_b = int(input("entrez␣la␣valeur␣de␣b␣:␣"))
    d, u, v = BigIntMath.extendedGCD(val_a, val_b)
    print(f"resultat␣:␣gcd({val_a},␣{val_b})␣=␣{d}")
    print(f"coefficients␣de␣bezout␣:␣u␣=␣{u},␣v␣=␣{v}")

    print("test␣du␣chiffrement␣affine")
    a = int(input("entrez␣la␣cl  ␣'a'␣(␣premier␣avec␣n)␣:␣"))
    b = int(input("entrez␣la␣cl  ␣'b'␣:␣"))
    n = 26
    if BigIntMath.modInverse(a, n) is None:
        print("␣a␣n'est␣pas␣inversible␣modulo␣26␣Le␣d chiffrement␣est␣
            impossible.")
    else:
        cipher = AffineCipher(a, b, n)
        message = input("entrez␣le␣message␣␣␣:␣")
        chiffre = cipher.encrypt(message)
        dechiffre = cipher.decrypt(chiffre)
        print(f"message␣chiffr  ␣:␣{chiffre}")
        print(f"message␣d  chiffr  ␣:␣{dechiffre}")
```

Méthode d'exécution :

- Le programme demande deux nombres pour tester l'algorithme d'Euclide étendu et affiche le pgcd et les coefficients.
- Il demande ensuite les clés `a` et `b` pour le chiffrement affine.
- Si `a` n'est pas inversible modulo 26, le déchiffrement est impossible.
- Sinon, le message est chiffré et déchiffré, puis les résultats sont affichés.
- L'utilisateur interagit en saisissant les valeurs et voit immédiatement le résultat.

```python
class BigIntMath:
    @staticmethod
    def extendedGCD(a, b):
        r, rr = a, b
        u, v = 1, 0
        uu, vv = 0, 1

        while rr != 0:
            q = r // rr
            r, rr = rr, r - q * rr
            u, uu = uu, u - q * uu
            v, vv = vv, v - q * vv

        return r, u, v

    @staticmethod
    def modInverse(a, n):
        d, u, v = BigIntMath.extendedGCD(a, n)

        if d != 1:
            return None

        return u % n

class AffineCipher:
    def __init__(self, a, b, n=26):
        self.a = a
        self.b = b
        self.n = n
        self.a_inv = BigIntMath.modInverse(a, n)

    def encrypt(self, message):
        res = ""
        for char in message.upper():
            if 'A' <= char <= 'Z':
                m = ord(char) - ord('A')
                c = (self.a * m + self.b) % self.n
                res += chr(c + ord('A'))
            else:
                res += char
        return res

    def decrypt(self, crypte):
        res = ""
        for char in crypte.upper():
```

```python
            if 'A' <= char <= 'Z':
                c = ord(char) - ord('A')
                m = (self.a_inv * (c - self.b)) % self.n
                res += chr(m + ord('A'))
            else:
                res += char
        return res


if __name__ == "__main__":

    print("␣test␣euclide␣")
    val_a = int(input("entrez␣la␣valeur␣de␣a␣:␣"))
    val_b = int(input("entrez␣la␣valeur␣de␣b␣:␣"))

    d, u, v = BigIntMath.extendedGCD(val_a, val_b)
    print(f"resultat␣:␣gcd({val_a},␣{val_b})␣=␣{d}")
    print(f"coefficients␣de␣bezout␣:␣u␣=␣{u},␣v␣=␣{v}")


    print("test␣du␣chiffrement␣affine")
    a = int(input("entrez␣la␣cl  ␣'a'␣(␣premier␣avec␣n)␣:␣"))
    b = int(input("entrez␣la␣cl  ␣'b'␣:␣"))
    n = 26


    if BigIntMath.modInverse(a, n) is None:
        print("␣a␣n'est␣pas␣inversible␣modulo␣26␣Le␣d chiffrement␣est␣
            impossible.")
    else:
        cipher = AffineCipher(a, b, n)
        message = input("entrez␣le␣message␣␣␣:␣")

        chiffre = cipher.encrypt(message)
        dechiffre = cipher.decrypt(chiffre)

        print(f"message␣chiffr  ␣:␣{chiffre}")
        print(f"message␣d  chiffr  ␣:␣{dechiffre}")
```

0,8 / 1

No theoretical perspective!